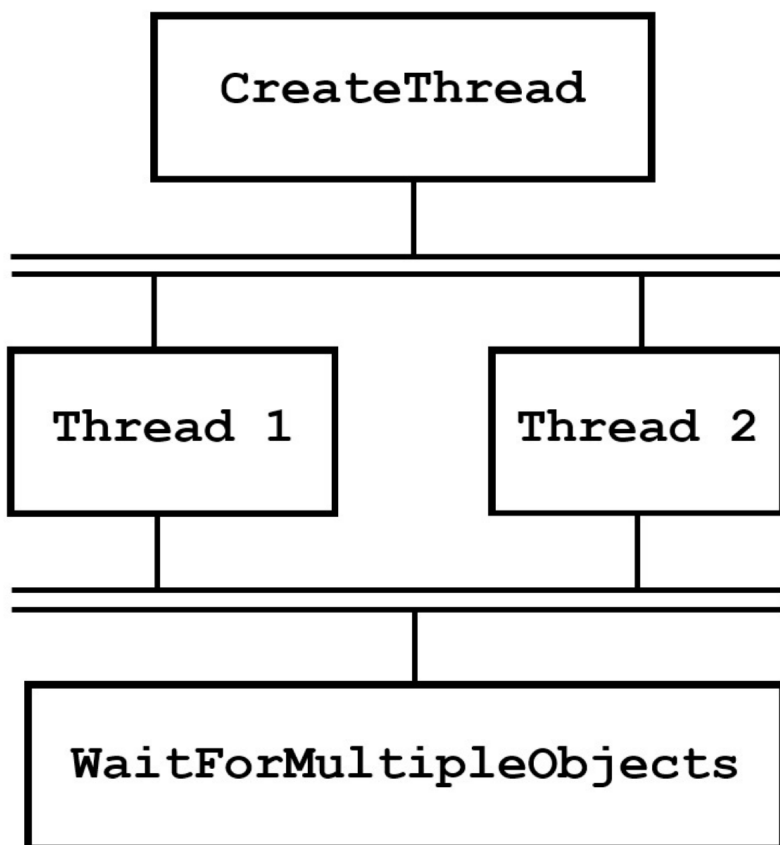


Валентин Юльевич Арьков

Организация параллельных потоков. Часть 1

Лабораторный практикум



Валентин Юльевич Арьков

Организация параллельных потоков. Часть 1

Лабораторный практикум

Издательские решения
По лицензии Ridero
2020

УДК 62
ББК 3
А89

Шрифты предоставлены компанией «ПараТайп»

Арьков Валентин Юльевич

А89 Организация параллельных потоков. Часть 1 : Лабораторный практикум / Валентин Юльевич Арьков. — [б. м.] : Издательские решения, 2020. — 46 с.
ISBN 978-5-4498-3657-1

В данной лабораторной работе рассматриваются основы организации параллельных потоков с помощью стандартных вызовов операционной системы.

В работе используется бесплатная интегрированная среда разработки. Приводятся примеры программ на языке Си.

**УДК 62
ББК 3**

12+

В соответствии с ФЗ от 29.12.2010 №436-ФЗ

ВВЕДЕНИЕ

Большинство современных процессоров в настольных компьютерах и мобильных устройствах — многоядерные. Чтобы эффективно использовать такие вычислительные ресурсы, необходимо составлять параллельные программы. Для этого можно задействовать параллельные потоки либо параллельные процессы.

В данной работе рассматривается технология параллельного программирования потоков **Microsoft Windows** встроенными средствами, без специальных библиотек. Для составления и компиляции программ на Си используется бесплатная интегрированная среда разработки **Microsoft Visual Studio Community Edition**.

Общие теоретические сведения об организации параллельных вычислений можно найти в соответствующей литературе [1–6]. В данной работе мы рассматриваем только основы организации параллельных потоков.

1. ОБЩИЕ СВЕДЕНИЯ О РАБОТЕ

Целью работы является изучение технологии программирования параллельных потоков **Microsoft Windows**.

Основные этапы выполнения работы:

- Процессы и потоки
- Процессоры и ядра
- Компилятор
- Параллельные потоки
- Состояние процесса
- Время выполнения
- Ускорение и эффективность
- Привязка задач к ядрам
- «Гонка» за доступ к данным
- Критическая секция

По результатам выполнения работы оформляется отчёт.

1.1. СОСТАВЛЕНИЕ И ОФОРМЛЕНИЕ ОТЧЁТА

Отчёт оформляется в виде рабочей книги **Microsoft Excel**.

Состав отчёта:

- Титульный лист
- Оглавление
- Остальные листы

Для удобства навигации вкладки листов нумеруем.

В оглавлении делаем ссылку на каждый лист.

Подробнее оформление отчёта в пакете **Excel** описано в работе [7].

Задание. Создайте файл отчёта и оформите титульный лист и оглавление.

1.2. КЛЮЧЕВЫЕ ОПРЕДЕЛЕНИЯ

Перед началом выполнения работы необходимо определиться с основными терминами.

Задание. Выясните, что означают перечисленные термины:

- прикладная программа (application);
- алгоритм программы
- исходный текст программы;
- компилятор;
- вычислительный процесс (process);
- поток исполнения / выполнения (thread);
- многопоточность;
- центральный процессор (CPU);
- ядро микропроцессора (Core);
- HyperThreading;
- виртуальный процессор.

Заметим, что независимо от физической конфигурации компьютера, операционная система представляет все вычислительные устройства как виртуальные процессоры.

Задание. Запустите **Task Manager (Диспетчер задач)** и определите следующие параметры:

- число виртуальных процессоров;
- размер доступной оперативной памяти;
- число выполняемых процессов;
- число потоков — общее и в каждом процессе;
- уровень загрузки процессора в целом и каждого виртуального процессора.

2. ОРГАНИЗАЦИЯ ВЫЧИСЛИТЕЛЬНЫХ ЭКСПЕРИМЕНТОВ

2.1. КОНФИГУРАЦИЯ КОМПЬЮТЕРА

Возможности улучшения вычислительной производительности компьютера ограничены его конфигурацией. Поэтому нам предстоит познакомиться со своим компьютером.

Задание. Ознакомьтесь с конфигурацией компьютера и параметрами системы с помощью следующих средств:

- свойства компьютера;
- диспетчер устройств;
- параметр окружения NUMBER OF PROCESSORS;
- сайт производителя процессора;
- диспетчер задач.

Опишите в отчёте конфигурацию и состояние системы:

- процессор;
- разрядность процессора;
- число ядер;
- поддержка HyperThreading;
- версия операционной системы;
- разрядность операционной системы;
- число выполняемых потоков на ядро;
- число виртуальных процессоров;
- объём занятой оперативной памяти;
- объём виртуальной памяти
- наличие подкачки.

2.2. СРЕДА РАЗРАБОТКИ

Данная работа выполняется с использованием среды разработки **Microsoft Visual Studio**. Далее рассматриваются примеры программ на языке программирования **C**, хотя компилятор поддерживает также и **C++**.

В работе используется бесплатная версия среды разработки: **Community Edition**.

Дистрибутив находится на сайте

VisualStudio.com.

Рекомендуется использовать английскую версию среды программирования.

Обратите внимание на название каталога, в который установлен компилятор.

В случае 32-разрядной ОС можно установить только 32-разрядные приложения. Каталог для установки:

Program Files.

В случае 64-разрядной ОС можно установить как 32-разрядные, так и 64-разрядные приложения. Каталоги для установки — соответственно:

Program Files (x86)

Program Files.

Далее мы будем создавать именно 32-разрядные приложения, причём работающие в командной строке.

Задание. Проверьте наличие компилятора на компьютере. Если компилятор установлен, запустите его. Если компилятор отсутствует, установите его.

Опишите в отчёте параметры компилятора:

- версия;
- язык интерфейса;
- разрядность.

3. ПРИЁМЫ ПРОГРАММИРОВАНИЯ

3.1. ПРОГРАММА HELLO, WORLD!

Hello, World — это первая программа для знакомства со средой разработки и процедурой компиляции.

Запустите **Visual Studio**. Создайте новый проект.

File — New — Project.

Выберите создание консольного приложения:

Create a new project — Empty project — Next.

Укажите название проекта:

Project name.

Каталог для хранения проекта:

Location.

Ставим галочку в строке:

Place solution and project in the same directory.

Нажмите кнопку

Create.

Добавьте файл с исходным текстом к проекту:

Solution Explorer — Source Files — [Контекстное меню] — Add — New Item.

В окне

Add New Item

установите

Visual C++ — Code — C++ File (.cpp).

Укажите название файла и его расположение. Нажмите кнопку **Add**.

Введите текст программы (рис. 3.1).

```
#include <stdio.h>
void main()
{
    printf("Hello, world!\n");
}
```

Рис. 3.1. Программа **Hello, World**

Сохраните файл, нажав комбинацию клавиш:

Ctrl + S.

Установите конфигурацию проекта

Release;

x86.

Чтобы скомпилировать исполняемый файл, вызываем в верхнем меню

Build – Solution.

Для запуска программы на выполнение используем комбинацию клавиш:

[Ctrl + F5].

Сохранение файла вызывается с помощью следующей комбинации клавиш:

[Ctrl + S].

Рекомендуем создавать проекты в подкаталогах одного рабочего каталога. Названия проектов, файлов и каталогов должны быть короткими и информативными. Чем короче путь к файлу, тем легче будет до него добраться в командной строке.

Задание. Ознакомьтесь с историей программы «**Hello, World**» на сайте **Википедии**. Выясните, как правильно переводится эта фраза на русский язык.

Задание. Прочитайте в **Википедии** статью **Консоль**. Выясните, что означает слово «консоль».

Задание. Создайте проект и запустите программу на выполнение:

- Создайте новый проект.
- Добавьте файл типа *.C.
- Введите исходный текст программы.
- Скомпилируйте проект и запустите программу.
- Скомпилируйте проект.
- Запустите программу из среды разработки.
- Запустите программу из Проводника Windows.

Для работы в командной строке нужно запустить командное окно:

[Win + R] — cmd — OK.

Чтобы перейти на нужный диск, введите букву и двоеточие. Например, для перехода на диск D:, нужно ввести

d:

Переход между каталогами — команда **cd**.

Чтобы подняться один на уровень выше по файловому дереву, используем команду

cd..

Просмотр содержимого каталога:

dir.

Для запуска программы на выполнение введите название программы

Задание. Запустите командную строку, перейдите в каталог своей программы и запустите её на выполнение.

3.2. ДИАПАЗОНЫ ЗНАЧЕНИЙ ПЕРЕМЕННЫХ

В нашей работе мы будем иметь дело с большими целыми числами — счётчиками и суммами. Поэтому при объявлении типа переменной нужно учитывать максимальное значение, которое можно будет записать в эту переменную.

Диапазон значений переменной зависит от количества байт. Стандартные целые типы используют 1, 2, 4 или 8 байт.

Кроме количества байтов, нужно учитывать наличие знака:

- числа со знаком (signed)
- числа без знака (unsigned).

Зная эти параметры, можно приблизительно оценить диапазон возможных значений. Для грубых оценок можно считать, что 2^{10} приблизительно равно 10^3 .

Задание. Составьте список целых типов и оцените диапазон их значений.

3.3. ПАРАМЕТРЫ КОМАНДНОЙ СТРОКИ

Программу (исполняемый файл типа ***.EXE**) обычно запускают на выполнение несколькими способами:

- среда разработки;
- Проводник Windows;
- файловый менеджер типа FAR.

При запуске программы на выполнение из **Проводника** обычно ограничиваются указанием имени исполняемого файла.

Задание. Запустите свою программу тремя перечисленными способами.

Командная строка — это один из первых видов компьютерных интерфейсов. Он может также упоминаться под другими названиями:

- командное окно;
- интерпретатор командной строки;
- командный процессор;
- command prompt.

При запуске программы в командной строке у нас появляются дополнительные возможности. После имени исполняемого файла `file.exe` можно дополнительно указать любые параметры `param1`, `param2`, `param3`, напечатав их через пробел:

file param1 param2 param3.

Расширение имени файла **EXE** можно не указывать.

В данной работе мы будем использовать параметры (аргументы) командной строки. Эти параметры попадают «внутри»

программы и доступны для использования во время выполнения программы.

Чтобы получить доступ к параметрам запуска программы в командной строке, будем использовать аргументы основного модуля программы:

main (int argc, char *argv []);

argc (argument count) — количество параметров командной строки;

argv [] (argument values) — массив указателей на строки (параметры командной строки).

Скомпилируем программу. Пусть этот файл называется **prog-arg. exe**. Мы запускаем его на выполнение в командной строке следующим образом:

prog-arg 1 param 23 аргумент 16.

Нулевой параметр **argv [0]** содержит имя запущенной программы. Если параметров нет, то мы получим только имя запущенной программы.

Задание. Измените текст программы так, чтобы получить параметры командной строки через аргументы **main** и вывести их на экран. Запустите программу из среды разработки, затем из командного окна с параметрами.

Проверьте, как работает вывод на экран параметров в виде чисел и текста на английском и на русском языках.

3.4. ЛОКАЛИЗАЦИЯ ВЫВОДА НА ЭКРАН

Для вывода на экран мы используем функцию

printf.

Экран (в текстовом режиме) — это стандартное устройство вывода.

Чтобы в консоли выводить русский текст, добавим пару строк внаше программу.

Подключаем заголовочный файл локализации **locale. h:**

#include <locale. h>

Затем вызываем функцию **setlocale:**

setlocale (LC_ALL, «Rus»).

Пример программы представлен на рис. 3.2:

```
#include <stdio.h>
#include <locale.h>
void main()
{
    setlocale(LC_ALL, "Rus");
    printf("Говорим по-русски\n");
}
```

Рис. 3.2. Локализация стандартного вывода

Задание. Создайте и запустите программу, представленную на рис. 3.2.

Задание. Добавьте локализацию в программу вывода параметров командной строки. Убедитесь, что программа корректно обрабатывает параметры на русском языке.

3.5. ЧИСЛА В КОМАНДНОЙ СТРОКЕ

Через командную строку можно ввести число и использовать его в работе программы.

Для этого мы добавим добавляем заголовочный файл `stdlib. h`:

#include <stdlib. h>.

Затем вызываем функцию

int atoi (char *str).

Чтение из командной строки

В зависимости от типа переменной преобразование строки в целое число выполняется следующими функциями:

```
int atoi (const char *str);  
long int atol (const char *str);  
long long atoll (const char *str).
```

Этой приём нам понадобится для быстрого изменения параметров работы программы. Мы указываем параметр командной строки и преобразуем его в число. Полученное число записываем в переменную, которую используем для управления работой программы.

Задание. Добавьте в программу вызов функции преобразования строки в число. Выведите полученное число на экран.

4. ПОСЛЕДОВАТЕЛЬНЫЕ ВЫЧИСЛЕНИЯ

4.1. ПРОГРАММА СУММИРОВАНИЯ

В наших экспериментах мы будем находить сумму выбранного количества единиц (рис. 4.1).

```
#include <stdio.h>
void main()
{
    int Ns = 1000000;
    int S = 0;
    int i;
    for ( i = 0; i < Ns ; i++ )
        S = S + 1;
    printf("S  = %d\n", S);
    printf("Ns = %d\n", Ns);
}
```

Рис. 4.1. Последовательное суммирование

При назначении типов переменных учитываем диапазон возможных значений.

Задание. Составьте программу для нахождения суммы единиц. Запустите программу и убедитесь в правильности вычислений.

Задание. Составьте программу суммирования с вводом ко-

личества единиц через параметры командной строки. Программа должна вначале вывести заданное значение, затем найти сумму и вывести значение полученной суммы.

Задание. Введите количество единиц, выходящее за пределы диапазона значений. Обратите внимание на поведение программы.

4.2. ВРЕМЯ РАБОТЫ ПРОГРАММЫ

Чтобы сравнивать скорость работы программ между собой, нам нужно будет определить время выполнения программы. В результате мы сможем вычислить показатели эффективности распараллеливания.

Для измерения времени нам понадобится следующая функция:

clock_t clock (void).

Эта функция не имеет входных параметров. Она просто сообщает нам, сколько тактов прошло с начала работы программы. Тип возвращаемого значения определён как

clock_t.

Мы должны задать вспомогательные переменные именно такого типа.

Чтобы пересчитать количество тактов в секунды, нужно разделить количество тактов на значение следующей константы:

CLOCKS_PER_SEC.

Чтобы использовать функцию **clock**, в начале программы нужно подключить библиотеку, указав заголовочный файл:

time. h.

Пример программы приводится на рис. 4.2.

Исходные данные для расчёта времени — целые числа. После деления может получиться дробное число. Чтобы найти результат в виде дробного числа, придётся в формуле переопределить тип переменных.

Зададим формат вывода на экран для дробного числа так, чтобы получить десять знаков после запятой:

%5.10f.

```
#include <stdio.h>
#include <time.h>
void main()
{
    int Ns = 1000000;
    int S = 0;
    int i;
    float dT;
    clock_t t1, t2;
    t1 = clock();
    for ( i = 0; i < Ns; i++ )
        S = S + 1;
    t2 = clock();
    dT = (float) (t2 - t1) / CLOCKS_PER_SEC;
    printf("S = %d\n", S);
    printf("Ns = %d\n", Ns);
    printf("dT = %5.10f s\n", dT);
}
```

Рис. 4.2. Измеряем время выполнения

Задание. Составьте программу суммирования с определением времени работы.

Задание. Измените программу так, чтобы вводить количество единиц в командной строке при запуске программы.

Задание. Запустите составленную программу с разным значением параметра **Ns** и укажите в отчёте время выполнения.

Задание. Подберите количество прямоугольников так, чтобы время выполнения программы составляло от 10 до 20 секунд.

4.3. ПАКЕТНАЯ ОБРАБОТКА

В предыдущем разделе мы запускали программу вручную. И результаты работы тоже записывали вручную. При большом количестве прогонов программы ручная работа становится неэффективной.

Чтобы выполнять рутинные действия в автоматическом режиме, мы будем использовать пакетный файл. Это текстовый файл с расширением ***.BAT**. В таком файле записаны команды операционной системы.

В простейшем варианте это будет несколько одинаковых строк с одной и той же командой запуска нашей программы. При выполнении пакетного файла операционная система запустит нашу программу.

Задание. Составьте пакетный файл для нескольких запусков программы.

4.4. ЗАГРУЗКА В EXCEL

Следующая проблема — это запись результатов в отчёт. Мы сформируем файл в формате **CSV**, то есть значения с разделителем:

Comma Separated Values.

В английском варианте разделитель полей — запятая, а разделитель целой и дробной частей — точка. Команда вывода на экран в языке Си тоже использует точку как десятичный разделитель. Нам остаётся организовать вывод полученных чисел в одну строку через запятую — при каждом прогоне программы. Далее мы переходим на следующую строку.

Вот пример вывода результатов в одну строку:

```
printf («%d», S)
```

```
printf («%d\n», Ns)
```

Чтобы формат вывода легче читался на экране, мы выводим каждое число по отдельности.

После нескольких прогонов мы получим несколько строк.

Если вставлять результаты в отчёт вручную, придётся копировать через буфер обмена. Это опять долго и сложно.

Для полной автоматизации работы мы просто перенаправим вывод текста и значений вместо экрана в файл. Для перенаправления стандартного вывода используется символ `>`. Пример перенаправления вывода в файл:

```
batch.bat> file. txt.
```

Кроме того, в начале файла нам нужно отключить вывод на экран каждой команды:

```
@echo off.
```

Далее идёт строка заголовков столбцов — тоже через запятую (рис. 4.3).

```
@echo off  
echo S,Ns,dT  
ParBat  
ParBat  
ParBat  
ParBat  
ParBat
```

Рис. 4.3. Содержимое пакетного файла

После прогона и получения текстового файла мы можем загрузить его в пакет Excel. Мы вызываем пункт меню

File – Open.

Затем указываем, что формат файла — текстовый:

Text Files (*.prn, *.txt, *.csv).

Далее с помощью **Мастера импорта** настраиваем загрузку текстового файла с разделителями и заголовками. Указываем,

что десятичный разделитель — точка.

Получаем готовую таблицу с результатами прогонов. И эти результаты уже можно обработать с помощью того же пакета Excel.

Задание. Настройте вывод программы в одну строку через запятую. Создайте пакетный файл по образцу рис. 5. Перенаправьте вывод в текстовый файл. Загрузите файл в Excel.

Задание. Проведите исследование эффективности распараллеливания, задавая разные значения для следующих параметров:

- количество единиц для суммирования;
- количество параллельных потоков;
- количество использованных виртуальных процессоров.

Представьте результаты исследования в виде таблиц и графиков.

5. ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ

5.1. СОЗДАНИЕ ПОТОКА

По умолчанию в любом процессе есть один главный поток. Дополнительные потоки программист создаёт «вручную».

Чтобы создать новый поток, используем функцию

CreateThread.

Это готовый, системный, библиотечный вызов операционной системы. Мы используем параметры по умолчанию. Нам понадобится указать только имя подпрограммы, которая будет выполняться в новом потоке (рис. 5.1). В нашем примере это **Thread2**.

```
#include <windows.h>
#include <stdio.h>
void WINAPI Thread2()
{
    printf("Thread Two\n");
    while(1);
}
void main()
{
    CreateThread(0, 0, (LPTHREAD_START_ROUTINE)Thread2, 0, 0, 0);
    printf("Main - Thread One\n");
    getchar();
}
```

Рис. 5.1. Создание потока

Задание. Создайте и запустите программу по созданию второго потока. Запустите **Диспетчер** задач и убедитесь, что в ней присутствует два потока.

Задание. Закомментируйте строку с бесконечным циклом

while (1) и запустите программу на выполнение. Запустите **Диспетчер задач** и обратите внимание на количество потоков.

5.2. ПРИВЯЗКА ПРОЦЕССА К ЯДРАМ

На многоядерном процессоре можно назначить определённые ядра (виртуальные процессоры) для выполнения выбранного процесса. Для этого в Диспетчере задач нужно выбрать нужный процесс, вызвать контекстное меню правой кнопкой мыши и выбрать пункт **Задать соответствие (Set Affinity)**.

В разных версиях Windows для задания соответствия используются разные вкладки **Диспетчера задач**:

- Windows 7 — вкладка **Процессы**;
- Windows 10 — вкладка **Details**.

В наших экспериментах можно вставить в начале программы ожидание ввода символа с клавиатуры:

getchar () или **__getch ()**.

После запуска программы она будет ожидать ввода. В этот момент мы изменяем привязку к ядрам. Затем нажимаем **Enter** на клавиатуре. Далее наша программа будет выполняться на указанных ядрах.

Задание. Запустите **Диспетчер задач** и обратите внимание на загрузку ядер. Задайте привязку своего процесса к одному ядру или нескольким ядрам. Обратите внимание на изменение загрузки ядер.

5.3. ПАРАЛЛЕЛЬНЫЕ ПОТОКИ И СИТУАЦИЯ ГОНКИ

Далее мы создадим программу параллельного суммирования. Для знакомства с проблемами распараллеливания будем находить сумму выбранного количества единиц. Затем задача будет постепенно усложняться.

Параллельное обращение к общей ячейке памяти (глобальной переменной) может приводить к нарушению целостности

данных. Один поток переписывает содержимое ячейки, не дожидаясь завершения операции другим потоком, то есть фактически «затирает» чужие результаты. Это так называемые «гонки», или соревнование потоков за доступ к данным. Английское название Data Race.

Мы будем изучать ситуацию «гонки» на примере параллельного суммирования (рис. 5.2). Каждый поток в цикле прибавляет единицы к глобальной переменной **S**. Мы объявляем её до начала модулей программы. Поэтому любой поток может прочитать и записать переменную **S**.

Если запустить такую программу несколько раз, результаты работы будут непредсказуемыми, случайными.

В текст программы нужно добавить определение времени расчётов.

```
#include <windows.h>
#include <stdio.h>
#define NTh 20
#define Ns 10000000
int S = 0;
void WINAPI ThreadProc()
{
    for (int i=0; i<Ns; i++)
        S = S + 1;
}
void main()
{
    HANDLE Thread [NTh];
    for (int i=0; i<NTh; i++)
        Thread[i] = CreateThread(0, 0, (LPTHREAD_START_ROUTINE)
ThreadProc, 0, 0, 0);
    WaitForMultipleObjects(NTh,Thread,TRUE,INFINITE);
    printf("S = %d\nError = %d\n",S,Ns*NTh-S);
}
```

Рис. 5.2. Параллельное суммирование

Задание. Составьте программу параллельного суммирования (рис. 5.2) и запустите её на выполнение. Откройте **Диспетчер задач** и обратите внимание на количество потоков в процессе,

а также на уровень загрузки виртуальных процессоров. Запустите программу 5 раз и запишите в отчёт продолжительность работы программы, значение суммы и ошибки.

5.4. КРИТИЧЕСКАЯ СЕКЦИЯ

Чтобы не допустить ситуацию гонки, используется синхронизация параллельных вычислений с помощью критической секции. Английское название — critical section.

Критическая секция — это часть параллельной программы, которую можно выполнять только последовательно, в одном потоке. Остальные потоки должны дожидаться завершения работы в этой частью кода, чтобы выполнить свою работу.

Вот этапы работы с критической секцией (К.С.):

Этап 1. Объявление переменной.

До начала основного модуля программы объявляем глобальную переменную:

CRITICAL_SECTION CS.

Это своего рода «указатель» на объект типа К.С.

Этап 2. Инициализация К. С.

В основном модуле программы инициализируем критическую секцию:

InitializeCriticalSection (&CS).

Начиная с этого момента, мы можем её использовать.

Этап 3. Начало К. С.

Мы объявляем, что входим в критическую секцию:

EnterCriticalSection (&CS).

Все команды, начиная с этого момента, могут выполняться только в одном потоке.

Этап 4. Конец К. С.

Объявляем, что выходим из критической секции

LeaveCriticalSection (&CS).

Дальнейшие команды снова можно выполнять в любом количестве параллельных потоков.

Этап 5. Удаление К. С.

В конце программы удаляем критическую секцию:

DeleteCriticalSection (&CS).

После этой команды мы не больше используем критическую секцию.

Пример программы параллельного суммирования с использованием К.С. приводится на рис. 5.3.

```
#include <windows.h>
#include <stdio.h>
#define Nth 20
#define Ns 1000000
int S = 0;
CRITICAL_SECTION CS;

void WINAPI ThreadProc()
{
    int i;
    for (i=0; i<Ns; i++)
    {
        EnterCriticalSection(&CS);
        S = S + 1;
        LeaveCriticalSection(&CS);
    }
}

void main()
{
    int i;
    HANDLE Thread [Nth];
    InitializeCriticalSection(&CS);
    for (i=0; i<Nth; i++)
        Thread[i] = CreateThread(0, 0,
            (LPTHREAD_START_ROUTINE) ThreadProc, 0, 0, 0);
    WaitForMultipleObjects(Nth, Thread,
        TRUE, INFINITE);
    DeleteCriticalSection(&CS);
    printf("S = %d\n", S);
}
```

Рис. 5.3. Использование критической секции

Задание. Составьте программу параллельного суммирования с использованием критических секций (рис. 5.3). Запустите скомпилированную программу на выполнение несколько раз и сравните результаты вычислений.

5.5. ВЗАИМОДЕЙСТВИЕ ПОТОКОВ

С помощью критической секции мы обеспечили целостность данных. Теперь программа выполняется без ошибок. Доступ к общей переменной S происходит корректно.

Однако, при этом работа существенно замедлилась. Каждый поток обращает сяк общей переменной и блокирует работу остальных потоков. Поэтому мы не получаем ожидаемого ускорения работы. Необходимо изменить алгоритм так, чтобы потоки работали параллельно, но при этом не затирали чужие результаты. А для нахождения общей суммы придётся задействовать критическую секцию, но это будет небольшая доля всех вычислений.

Задание. Модифицируйте программу (рис. 5.3) так, чтобы получить приемлемое ускорение без потери точности расчетов, используя глобальные и локальные переменные.

5.6. ЭФФЕКТИВНОСТЬ РАСПАРАЛЛЕЛИВАНИЯ

При анализе работы параллельных программ используют два основных показателя (рис. 5.4):

- ускорение S_p ;
- эффективность E_p .

В этих формулах использованы следующие обозначения:

p — количество виртуальных процессоров (вычислительных устройств);

T_1 — время выполнения последовательной программы на одном виртуальном процессоре;

T_p — время выполнения параллельной программы на p виртуальных процессорах;

$$S_p = \frac{T_1}{T_p} \quad E_p = \frac{S_p}{p}$$

Рис. 5.4. Ускорение и эффективность

Для вычисления этих показателей нужно определить длительность выполнения последовательной и параллельной программ.

В идеальной ситуации ускорение должно быть равно количеству виртуальных процессоров (ядер). Например, для процессора с восемью ядрами можно ожидать ускорение вычислений в восемь раз.

Задание. Определите теоретические значения ускорения и эффективности вашего компьютера и сравните их с полученными в экспериментах.

6. ПАРАЛЛЕЛЬНОЕ ЧИСЛЕННОЕ ИНТЕГРИРОВАНИЕ

Мы рассмотрели основы организации параллельных вычислений. В качестве примера задачи для распараллеливания мы использовали нахождение суммы единичек. Теперь мы используем все наши предыдущие наработки и решим более сложную задачу.

Мы будем рассматривать задачу численного нахождения значения определённого интеграла методом прямоугольников. Результаты численных методов должны приближаться к теоретическим (аналитическим).

Мы будем оценивать точность и скорость вычислений, а также показатели эффективности распараллеливания.

6.1. ВАРИАНТЫ ЗАДАНИЙ

В задании необходимо найти значение определённого интеграла (рис. 6.1).

$$S = \int_A^B (a_3x^3 + a_2x^2 + a_1x + a_0)dx$$

Рис. 6.1. Определённый интеграл

Значения коэффициентов и пределы интегрирования для каждого варианта приведены в Таблице 6.1.

Таблица 6.1. Варианты заданий

Вариант	a_3	a_2	a_1	a_0	A	B
0	0,05	0,2	-10	200	-20	20
1	0,1	0,1	-10	200	-10	20
2	0,05	0,1	-10	100	-15	10
3	-0,1	0,5	-10	200	-20	5
4	0,05	0,3	-20	200	-5	20
5	0,03	0,3	-10	50	-20	10
6	0,02	0,2	-3	200	-20	5
7	0,05	0,1	-10	60	-15	5
8	0,04	0,3	-10	100	-5	20
9	0,05	0,5	-5	150	-20	20
10	0,06	0,3	-8	110	-10	20

Нулевой вариант используется для демонстрации.

Студент выбирает вариант по последней цифре номера зачётной книжки. Если последняя цифра – ноль, используем вариант 10.

Задание. Укажите в своём отчёте номер варианта и параметры задания.

6.2. ГРАФИК ФУНКЦИИ

Ознакомимся с подынтегральным выражением. Построим график этой функции в заданных пределах.

Для построения графика используем возможности пакета

Excel (рис. 6.2). Задаём столбец «иксов» и вычисляем столбец «игреков». Затем вставляем точечную диаграмму с гладкими кривыми.

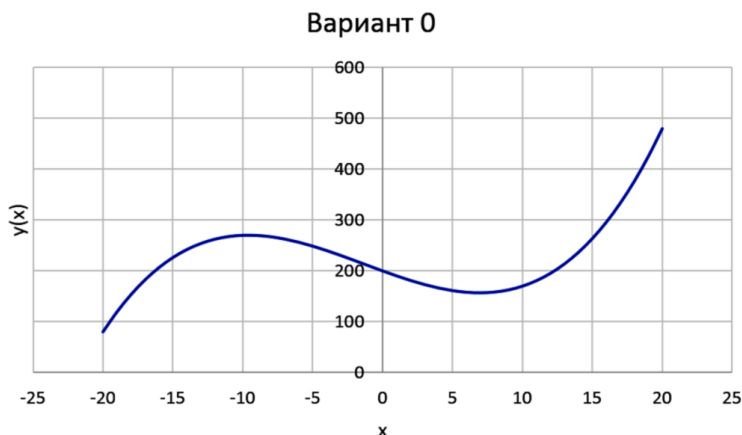


Рис. 6.2. График функции

Задание. постройте график своей функции в заданных пределах.

6.3. ГРУБАЯ ОЦЕНКА

Для начала попробуем оценить площадь фигуры, то есть площадь под графиком функции $f(x)$ в пределах значений аргумента от -20 до $+20$.

«Икс» меняется от -20 до $+20$, значит основание фигуры равно 40.

Высота — это среднее значение функции на интервале. Рассмотрим график нашей функции на рис. 6.2. Значения меняются от 100 до 300, а в конце диапазона возрастают до 500. Так что можно приблизительно оценить среднее значение $f(x) = 200$.

Умножаем основание на высоту и получаем грубую оценку площади:

$$S = 40 * 200 = 8000$$

Это очень приближённая оценка. Можно сказать, что это только ПОРЯДОК величины. Слово «порядок» в данном случае означает «степень десятки». В какую степень нужно возвести число 10, чтобы получить наше значение. То есть речь идёт о числах порядка десяти тысяч. Не о тысяче и не о ста тысячах. Это будет число «около десяти тысяч».

Грубые, предварительные, приблизительные оценки нужны для того, чтобы выявлять ещё более грубые ошибки. Например, если перепутали в расчётах плюс с минусом. Или не в ту степень число возвели.

Задание. Оцените среднее значение подынтегральной функции по графику $f(x)$.

Задание. Сделайте грубую, предварительную оценку значения своего интеграла.

6.4. АНАЛИТИЧЕСКОЕ РЕШЕНИЕ

Вначале определим точное теоретическое значение интеграла. Для этого найдем решение аналитически (рис. 6.4).

$$I = \int_a^b f(x)dx = F(b) - F(a)$$

Рис. 6.4. Аналитическое решение

Здесь нам потребуется найти первообразную $F(x)$ для подынтегрального выражения $f(x)$. Это функция $F(x)$, производная которой даёт нам $f(x)$, см. рис. 6.5.

$$f(x) = 0.05x^3 + 0.2x^2 - 10x + 200$$

$$F(x) = 0.05x^4/4 + 0.2x^3/3 - 10x^2/2 + 200x$$

Рис. 6.5. Нахождение первообразной

Далее находим разность значений первообразной на границах B и A:

$$S = F(20) - F(-20) = 9066,67$$

Расчёты можно сделать в пакете Excel (рис. 6.6).

Полученное значение очень близко к нашей грубой оценке 8000. Оба числа близки к 10000. Это и есть ПОРЯДОК величины. Разница не в десять раз. И даже не в два раза, а гораздо меньше. Похоже, что грубых оценок в наших аналитических выкладках не было.

	J	K	L	M	N	O
5						
6						
7						
8				x	F(x)	
9			B	20	4533,33	
10			A	-20	-4533,33	
11				F(B)-F(A)=	9066,67	
12						

Рис. 6.6. Аналитическое решение

Далее наши приближённые численные решения будем сравнивать с этим теоретическим значением.

Задание. Определите значение своего интеграла аналитически и сравните его с предварительной грубой оценкой.

6.5. МЕТОД ПРЯМОУГОЛЬНИКОВ

Численные методы используют приближённую замену сложной функции более простыми. В методе прямоугольников мы заменяем подынтегральную функцию на несколько смежных прямоугольников.

Диапазон значений аргумента (пределы интегрирования) мы разбиваем на некоторое количество отрезков одинаковой длины. Высота каждого прямоугольника равна значению подынтегральной функции в середине отрезка (рис. 6.7).

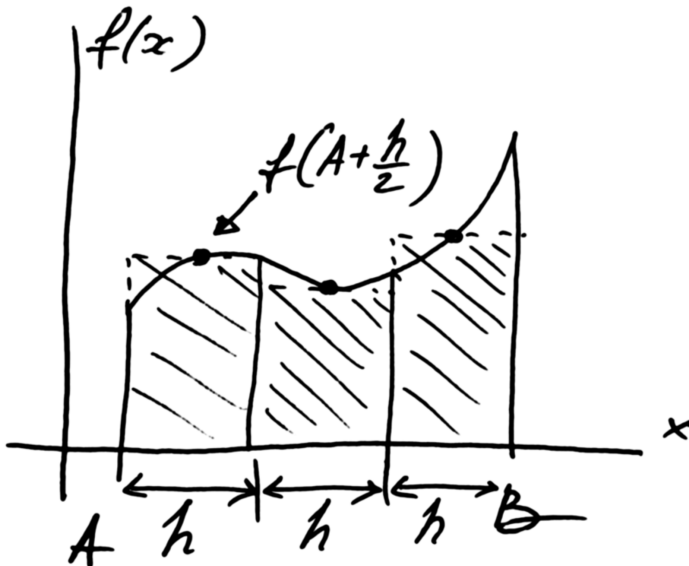


Рис. 6.7. Метод прямоугольников

Значение определённого интеграла равно площади под графиком функции. Эту площадь мы приближённо оцениваем как сумму площадей прямоугольников.

Основания всех прямоугольников основания равны. Поэтому

мы можем вначале сложить все значения функции, а затем умножить эту сумму на длину отрезка.

Для данного эксперимента мы используем вещественный тип `float`.

Задание. Составьте последовательную программу численного интегрирования, указав 100 отрезков. Используйте тип `float`. Сравните полученный результат с теоретическим.

Задание. Подберите количество отрезков так, чтобы время выполнения программы составило от 10 до 20 секунд.

6.6. ИССЛЕДОВАНИЕ ПОГРЕШНОСТЕЙ

Численные методы всегда дают приближённый результат. Вопрос только в том, какой уровень погрешности позволит работать с полученными результатами и решать практическую задачу — определить площадь, объём, нагрузку, деформацию изделия и тому подобное.

В нашем примере погрешность зависит, прежде всего, от количества прямоугольников. Нам предстоит исследовать эту зависимость и построить графики.

Задание. Запустите программу численного интегрирования с разным количеством прямоугольников. Организуйте ввод количества прямоугольников через параметры командной строки. При вычислениях используйте тип `float`. Используйте пакетный файл и сделайте по 10 запусков программы для каждого количества прямоугольников. Зафиксируйте в отчёте среднее время вычислений и уровень погрешности в процентах от теоретического значения.

Второй фактор, влияющий на уровень погрешности, — это тип переменной. В стандартном языке Си есть два типа переменных с плавающей точкой: `float` и `double`. Их ещё называют вещественными числами — в отличие от целых. Они обеспечивают разное количество значащих десятичных разрядов.

Задание. Выясните, какую точность представления обеспе-

чивают два вещественных типа.

Задание. Проведите эксперименты по численному интегрированию с типом **double**. Сравните уровень погрешности и время вычислений.

Каждый тип переменной имеет свои пределы возможных значений. Выход в большую и меньшую сторону приводит к потере результатов. К тому же, у нас есть положительные и отрицательные числа.

Задание. Выясните пределы возможных значений для обоих вещественных типов в положительной и отрицательной области значений.

Задание. Составьте тестовые программы и выясните, что происходит при выходе за пределы допустимых значений для обоих вещественных типов.

С увеличением числа прямоугольников погрешность вычислений должна уменьшаться. Однако, при слишком большом числе прямоугольников должно начаться снижение точности и рост погрешности (рис. 6.8). При сложении большого числа с маленьким возникает проблема потери точности. Маленькое число может просто выйти за границы разрядной сетки — это так называемое «обратное переполнение» или «потеря точности / значимости». В нашем случае накопленная сумма значений может оказаться гораздо больше, чем новое прибавляемое значение.

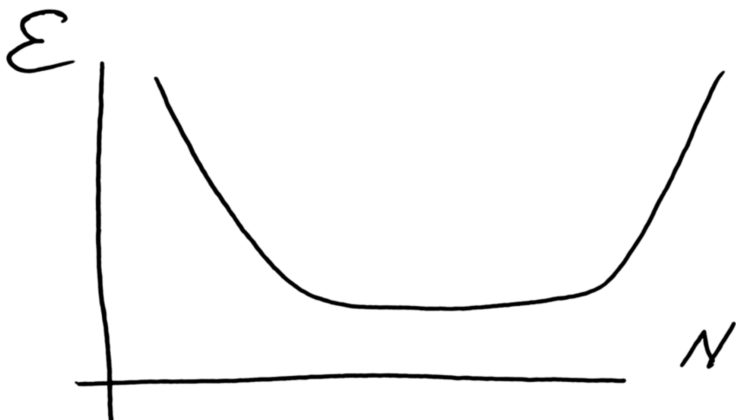


Рис. 6.8. Изменение погрешности вычислений

Можно грубо оценить число прямоугольников, при котором сумма половины значений будет значительно больше, чем одно отдельное, очередное значение $f(x)$. Другими словами, одно значение $f(x)$ должно выйти за пределы разрядной сетки — по сравнению с накопленной суммой.

Более точно можно найти среднее как отношение значения интеграла к интервалу «иксов». Это равносильно замене графика $f(x)$ на прямоугольник с таким же основанием и высотой, соответствующей среднему значению. Площади этих фигур будут равны (рис. 6.9).

Получаем среднее значение аналитически:

$$f_{cp} = S / (B - A) = 9066,67 / 40 = 227.$$

Напомним, что наша грубая оценка среднего дала значение 200. Цифры получились достаточно близкими.

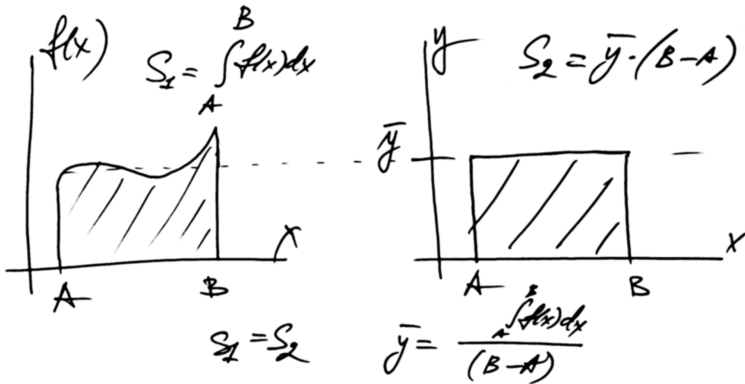


Рис. 6.9. Эквивалентный по площади прямоугольник

Задание. Найдите среднее значение $f(x)$ аналитически и сравните с грубой предварительной оценкой.

Представим себе ситуацию, когда сумма S половины значений функции $f(x)$ стала гораздо больше одного отдельного значения $f(x)$, которое к этой сумме надо прибавить:

$S \gg f(x)$.

Ситуация потери точности показана на рис. 6.10. В этом примере от всех разрядов $f(x)$ в общую сумму попадает только самый старший, а остальные теряются.

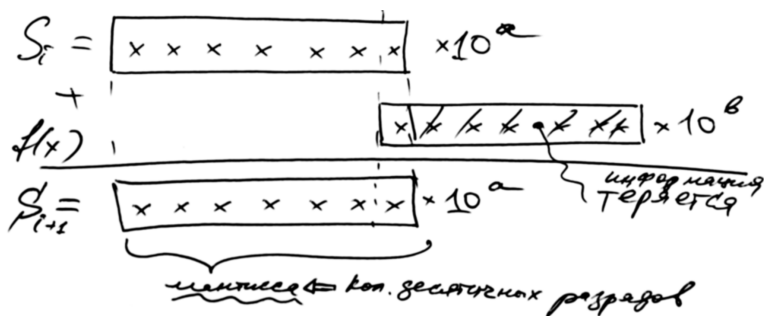


Рис. 6.10. Потеря точности

Примерно оценим сумму половины значений — как среднее $f(x)$, умноженное на половину количества прямоугольников:

$$S = f_{cp} * N / 2.$$

Далее можно оценить, при каком числе прямоугольников начинают теряться младшие разряды значения $f(x)$.

Задание. Оцените количество прямоугольников, при котором может начаться потеря точности для переменных типа `float` и `double`. Для этого определите, когда будет теряться 1, 2, 3 и т. д. младших разрядов. Оцените время вычислений и тип счётчика цикла.

Задание. Проведите эксперименты, чтобы обнаружить начало потери точности с обоими вещественными типами.

6.7. ПАРАЛЛЕЛЬНОЕ ИНТЕГРИРОВАНИЕ

Мы провели подготовительные работы. Можно переходить к экспериментам с параллельными программами. Будем задавать число потоков, равное числу ядер (виртуальных процессоров).

При параллельных вычислениях нужно раздать вычислительные задания разным потокам примерно поровну. Для этого можно использовать метод чередования. Первый поток получа-

ет первый отрезок, второй поток — второй отрезок и так далее до максимального номера ядра. Например, до восьмого. Затем мы идём по кругу: первый поток получает девятый отрезок, второй — десятый и так далее. Раздача отрезков продолжается до тех пор, пока не закончатся отрезки.

При организации параллельных вычислений мы просто сообщаем каждому создаваемому потоку его порядковый номер — от 1 до максимального. Затем каждый поток сам находит те значения, которые назначены именно ему.

Задание. Составьте и прогоните параллельные программы для численного интегрирования в соответствии с вариантом задания. Определите показатели эффективности распараллеливания. При возможности доведите программу до потери точности.

Дополнительные эксперименты — это использование части вычислительных ядер, доступных на данном компьютере. В простейшем случае мы запускаем программу, затем она ожидает нажатия клавиши. В этот момент мы задаём Affinity, то есть «соответствие» ядер нашему процессу через Диспетчер задач.

Задание. Прогоните параллельную программу, задавая разные ядра в разном количестве. Определите показатели эффективности распараллеливания.

Ещё одна дополнительная сложность при создании параллельных программ — это вмешательство операционной системы (ОС) и попытки оптимизировать работу компьютера в целом. При выполнении многопоточной программы ОС может использовать ядра так, чтобы обеспечить их равномерную загрузку. ОС будет перебрасывать потоки с одного ядра на другое. Это дополнительные затраты вычислительной мощности. Наша цель — обеспечить максимальную загрузку выбранных ядер. Загрузка должна быть 100% во всех экспериментах.

Для этого можно назначать привязку к ядрам как для всего процесса в целом, так и для каждого потока.

Привязкой к ядрам можно управлять с помощью следующих функций:

- **GetProcessAffinityMask;**
- **SetProcessAffinityMask;**
- **SetThreadAffinityMask.**

Маска привязки системы определяет все доступные ядра.

Маска привязки процесса **ProcessAffinityMask** — это набор битов, разрешающих выполнение текущего процесса на выбранных ядрах (виртуальных процессорах) — из списка доступных.

Маска привязки потока **ThreadAffinityMask** — это разрешение использовать указанные ядра текущим потоком — в пределах маски процесса.

Задание. Создайте тестовые программы для работы с привязкой процесса и потоков к ядрам.

Задание. Модифицируйте программу параллельного интегрирования так, чтобы каждый поток выполнялся только на одном ядре. Определите показатели эффективности распараллеливания.

ЛИТЕРАТУРА

1. **Воеводин В. В., Воеводин Вл. В.** Параллельные вычисления. — СПб.: БХВ-Петербург, 2004. — 608 с.

2. **Воеводин Вл. В., Жуматий С. А.** Вычислительное дело и кластерные системы. — М.: МГУ, 2007. — 150 с.

Режим доступа:

<https://parallel.ru/info/parallel/cluster/>

3. **Антонов А. С.** Введение в параллельные вычисления: Методическое пособие. — М.: МГУ, 2002. — 70 с.

Режим доступа:

https://parallel.ru/news/antonov_intro.html

4. **Гергель В. П.** Теория и практика параллельных вычислений. — М.: Интуит, 2016. — 501 с.

5. **Гергель В. П., Фурсов В. А.** Лекции по параллельным вычислениям: Учебное пособие. — Самара: СГАУ, 2009. — 164 с.

6. **Гергель В. П.** Высокопроизводительные вычисления для многоядерных многопроцессорных систем: Учебное пособие. — Нижний Новгород: ННГУ, 2010. — 421 с.

7. **Арьков В. Ю.** Анализ распределения в Excel: Учебное пособие. — [б. м.]: Издательские решения, 2019. — 158 с.

Режим доступа:

https://ridero.ru/books/analiz_raspredeleniya_v_excel/

ОГЛАВЛЕНИЕ

Введение	3
1. Общие сведения о работе	4
1.1. Составление и оформление отчёта	4
1.2. Ключевые определения	5
2. Организация вычислительных экспериментов	6
2.1. Конфигурация компьютера	6
2.2. Среда разработки	7
3. Приёмы программирования	8
3.1. Программа Hello, World!	8
3.2. Диапазоны значений переменных	10
3.3. Параметры командной строки	11
3.4. Локализация вывода на экран	12
3.5. Числа в командной строке	13
4. Последовательные вычисления	15
4.1. Программа суммирования	15
4.2. Время работы программы	16
4.3. Пакетная обработка	18
4.4. Загрузка в Excel	18
5. Параллельные вычисления	21
5.1. Создание потока	21
5.2. Привязка процесса к ядрам	22
5.3. Параллельные потоки и ситуация гонки	22
5.4. Критическая секция	24
5.5. Взаимодействие потоков	27
5.6. Эффективность распараллеливания	27
6. Параллельное численное интегрирование	29
6.1. Варианты заданий	29
6.2. График функции	30
6.3. Грубая оценка	31
6.4. Аналитическое решение	32
6.5. Метод прямоугольников	34
6.6. Исследование погрешностей	35
6.7. Параллельное интегрирование	39

ЛИТЕРАТУРА	42
------------------	----

Валентин Юльевич Арьков

Организация параллельных потоков. Часть 1
Лабораторный практикум

Создано в интеллектуальной издательской системе Ridero

Арьков Валентин Юльевич — доктор технических наук, профессор кафедры автоматизированных систем управления Уфимского государственного авиационного технического университета. Опыт преподавания в вузе более 30 лет.

В данной лабораторной работе рассматриваются основы организации параллельных потоков с помощью стандартных вызовов операционной системы.

В работе используется бесплатная интегрированная среда разработки. Приводятся примеры программ на языке Си.

ISBN 978-5-4498-3657-1



9 785449 836571 >

Rideró

Rideró.ru — издай
книгу бесплатно!